

Built-in PlaceholderAPI (YaP)

YaPcore ships a **clean-room, clip-compatible** PlaceholderAPI so plugins that soft/hard-depend on HelpChat’s PlaceholderAPI work **without** installing `PlaceholderAPI.jar` from SpigotMC / Modrinth.

Item	Value
Product jar	<code>plugins/yap-placeholderapi.jar</code>
Plugin name (for <code>getPlugin</code> / depends)	PlaceholderAPI
Package	<code>me.clip.placeholderapi.*</code> (binary-compatible surface)
Built-in expansions	<code>player</code> , <code>server</code> (full upstream-style placeholder sets)
Extra expansions	Drop jars into <code>plugins/PlaceholderAPI/expansions/</code>
Admin	<code>/papi</code> full local command tree

Intentional product scope

YaP PlaceholderAPI is a **first-party, curated local-expansions** engine — not a partial stub and not a HelpChat eCloud mirror.

In scope	Out of scope (by design)
Parse API, built-ins <code>player</code> / <code>server</code>	HelpChat eCloud browse / download / update
Jar-drop external expansions	Bundling third-party expansion jars in release zips
First-party <code>%yap*_*%</code> from other YaP plugins	GPL eCloud coupling
<code>/papi</code> <code>parse\ list\ reload\ register\ dump\ ...</code>	

Operators curate expansions on disk. That is the supported delivery path.

Product rule

Do **not** also install HelpChat / clip PlaceholderAPI. Two jars both named `PlaceholderAPI` will conflict. YaP’s jar is the supported path.

```
gradle :placeholderapi-plugin:installIntoPlugins
# or: gradle shadowJar / assembleRelease
```

Local expansions workflow

1. Create (auto on first enable) or open `plugins/PlaceholderAPI/expansions/`
2. Copy a compatible expansion `.jar` into that folder
3. Run `/papi reload` or `/papi register <jar>`
4. Confirm with `/papi list` / `/papi info <id>`

```
plugins/PlaceholderAPI/expansions/
```

`/papi ecloud` explains this workflow (eCloud download UX is intentionally absent).

Commands

Command	Purpose
<code>/papi help</code>	Command list
<code>/papi parse <me\ --null\ player> <text...></code> <code>></code>	Parse placeholders
<code>/papi bcparse <target> <text...></code>	Broadcast parsed text
<code>/papi cmdparse <target> <text...></code>	Dispatch parsed text as a command
<code>/papi parserel <p1> <p2> <text...></code>	Relational placeholders
<code>/papi dump</code>	Local dump file + optional paste.helpch.at upload
<code>/papi list</code> / <code>info [id]</code> / <code>reload</code>	Expansion admin
<code>/papi register <jar></code> / <code>unregister <id></code>	Expansion folder admin
<code>/papi version</code>	Engine version
<code>/papi ecloud</code>	Points at local expansions folder workflow (no eCloud)

What works for other plugins

- `softdepend: [PlaceholderAPI]` / `depend: [PlaceholderAPI]`
- `Bukkit.getPluginManager().getPlugin("PlaceholderAPI") != null`
- `PlaceholderAPI.setPlaceholders(player, text)` (+ bracket / relational)
- `new MyExpansion().register()` extending `PlaceholderExpansion`
- Expansion register / unregister events
- `PlaceholderAPIPlugin.getAdventure()` (BukkitAudiences)
- `Msg`, `booleanTrue/False`, `getDateFormat()`, `getServerVersion()`
- Configurable / Cacheable / Cleanable / Taskable / Relational / VersionSpecific

Built-in placeholders

Player — name, displayname, uuid, health, *food*, *level/exp*, gamemode, world, *coords*, *yaw/pitch*, *biome*, direction, *ping/colored_ping*, *armor_*, *item_in_hand*, *permissions* (*has_permission_**), *potion effects*, *locale*, flight/sneak/sprint/dead flags, bed/compass, first/last join, and more.

Server — name, online / `online_<world>`, max_players, unique_joins, version/build, tps / colored tps, uptime, ram_, *whitelist*, *total_* entity/chunk counts, `time_*` patterns, countdown/countup helpers.

First-party expansions

Registered **in-process** by other YaP jars when installed (no jar drop needed):

Plugin	Identifier	Notes
yap-perms	yapperms	Permission / group placeholders
skills	yapskills	Skills / levels
factions	yapfactions	Faction data
guilds	yapguilds	Guild data
games	(game-specific)	Match / arena helpers
stacker	yapstacker	See STACKER.md

Ops extras

- **bStats** charts (YaP service id, not HelpChat's)
- **Update checker** — `check-updates` + optional `update-check-url`
- **Adventure** — `getAdventure()` for expansions that send components

Honesty bar

Clean-room compatibility engine — not a GPL port. Expansion delivery is **intentionally local and operator-curated**; eCloud download UX is out of scope. Everything else above is in-tree.

See also [PLUGIN_COMPAT.md](#) · [PLUGINS.md](#).